

Atividade Prática I:

Parte 1: Análise do Sistema

Ao analisar o código inicial, percebemos que a classe Pedido estava fazendo muita coisa. Ela era responsável por adicionar itens, calcular total, frete, desconto, atualizar estoque, processar pagamento, enviar notificação, gerar relatório e até salvar no banco de dados. Isso deixava a classe muito grande e difícil de entender e modificar.

Problemas Identificados:

1. **Coesão:** A classe Pedido tinha baixa coesão, pois acumulava responsabilidades demais. Era como um canivete suíço que fazia de tudo um pouco.
2. **Acoplamento:** Havia um alto acoplamento. A classe Pedido dependia diretamente de RelatorioService (criando uma instância interna) e BancoDeDados (chamando métodos =estáticos). Isso significa que se mudássemos algo em RelatorioService ou BancoDeDados , poderíamos ter que mexer em Pedido , o que não é legal.
3. **Ocultamento de Informação:** Muitos atributos da classe Pedido eram públicos (produtos , precos , quantidades , clienteNome , clienteEmail , clienteEndereco , total , frete , status). Isso permitia que qualquer parte do código acessasse e modificasse esses dados diretamente, o que pode causar inconsistências e bugs.
4. **Integridade Conceitual:** Não havia uma padronização clara. Por exemplo, os cálculos estavam espalhados pela classe Pedido , e a forma de lidar com itens do pedido era através de três listas separadas (produtos , precos , quantidades), o que dificultava a manipulação.

Parte 2: Refatoração Proposta

Para resolver esses problemas, decidimos dividir as responsabilidades e organizar melhor

o código. Nossa ideia foi criar classes menores e mais focadas, seguindo o princípio da responsabilidade única.

Modificações Realizadas:

1. Coesão:

O que foi alterado? Foram criadas classes específicas para lidar com responsabilidades que antes estavam todas na classe Pedido. Essas novas classes terminam em Service. Isso garante que cada serviço tenha uma responsabilidade única e clara.

Por que a nova solução é mais coesa? A solução ficou mais coesa pois as responsabilidades ficaram separadas em componentes especializados. Cada componente foca exclusivamente em sua lógica interna.

2. Acoplamento:

Quais dependências foram removidas? A criação de relatório dentro de pedido. O cliente ficava dentro do pedido, agora foi criada uma classe de entidade Cliente.

Onde foram usadas abstrações? Foi utilizada uma abstração em RelatorioService.

3. Ocultamento de Informação:

O que foi encapsulado?

Todos os atributos das classes que estavam como públicos foram alterados para privados.

Como o acesso aos dados foi controlado?

Atributos somente são acessíveis através de métodos getters agora e para alterar atributos é necessário utilizar métodos para essa finalidade.

4. Integridade Conceitual:

Como os nomes foram padronizados? Classes de serviço terminam com Service, Classes de entidade usam apenas o nome da entidade, Classes

referente a persistência de dados possuem a nomenclatura Repository.

Quais inconsistências foram removidas? Classe de entidade Pedido.